

WEEK 4: CUSTOMER ORDER MANAGEMENT SYSTEM

1. Requirement Analysis

Customer Order Management

The Auto Hub system needs to maintain customer order information and product-wise order details. The system stores important order details such as customer, order date, total amount, and order status.

The main objective of this module is to manage customer orders, track products ordered by customers, and generate customer purchase and order reports.

Relationship

- One customer can place many orders.
- One order can contain multiple products.
- The Customer_ID in the Orders table references the Customer table.
- The Order_ID in the Order_Details table references the Orders table.
- The Product_ID in the Order_Details table references the Product table.

2. Orders Table Creation

The Orders table was created to store customer order information. It contains Order ID, Customer ID, Order Date, Total Amount, and Order Status.

```
SQL> CREATE TABLE ORDERS (Order_ID VARCHAR2(10), Customer_ID VARCHAR2(10), Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));
```

```
Table created.
```

```
SQL> CREATE TABLE ORDER_DETAILS (Order_Detail_ID VARCHAR2(10), Order_ID VARCHAR2(10), Product_ID VARCHAR2(10), Quantity NUMBER, Price NUMBER(10,2));
```

```
Table created.
```

```
SQL> INSERT INTO ORDERS VALUES('2501','C-2001',To_Date('2026-03-12','YYYY-MM-DD'),9000.00,'Shipped');
1 row created.

SQL> INSERT INTO ORDERS VALUES('2502','C-2002',To_Date('2026-06-21','YYYY-MM-DD'),80000.00,'Delivered');
1 row created.
```

4. Primary Key Constraints

Primary key constraints were added to both tables.

- Order_ID was defined as the Primary Key of the Orders table.
- Order_Detail_ID was defined as the Primary Key of the Order_Details table.

5. Foreign Key Relationships

Foreign key relationships were established to connect the Orders and Order_Details tables with the Customer and Product tables.

- Customer_ID references the Customer table.
- Order_ID references the Orders table.
- Product_ID references the Product table.

This ensures that orders are associated with valid customers and products.

6. NOT NULL and CHECK Constraints

A NOT NULL constraint was applied to the Order Date to ensure that every order contains a valid order date.

A CHECK constraint was applied to Total Amount to ensure that the order amount is not negative.

```

SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_OrderDetails PRIMARY KEY (Order_Detail_ID);
Table altered.

SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_Orders_Customer FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Orders FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Product FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID);
Table altered.

SQL> ALTER TABLE ORDERS MODIFY Order_Date NOT NULL;
Table altered.

SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);
Table altered.

```

7. Insert Orders Values

Five order records were inserted into the Orders table. The records contain customer ID, order date, total amount, and order status.

```

SQL> INSERT INTO ORDERS VALUES('2503','C-2003',To_Date('2026-09-21','YYYY-MM-DD'),100000.00,'Delivered');
1 row created.

SQL> INSERT INTO ORDERS VALUES('2504','C-2004',To_Date('2026-01-18','YYYY-MM-DD'),300000.00,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES('2505','C-2005',To_Date('2026-05-27','YYYY-MM-DD'),250000.00,'Shipped');
1 row created.

```

```

SQL> INSERT INTO ORDERS VALUES('2501','C-2001',To_Date('2026-03-12','YYYY-MM-DD'),9000.00,'Shipped');
1 row created.

SQL> INSERT INTO ORDERS VALUES('2502','C-2002',To_Date('2026-06-21','YYYY-MM-DD'),80000.00,'Delivered');
1 row created.

```

Order detail records were inserted to maintain the products included in customer orders. The records contain order detail ID, order ID, product ID, quantity, and price.

```
SQL> INSERT INTO ORDER_DETAILS VALUES('O101','2501','P2001',15,28500.00);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('O102','2502','P2002',25,3499.00);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('O103','2503','P2003',8,42000.00);
1 row created.
```

9. Display Order and Order Details Records

The inserted records were displayed using the SELECT operation to verify that the order and order detail data was successfully stored in the tables.

```
SQL> SELECT * FROM ORDERS;
```

ORDER_ID	CUSTOMER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
2506	C-2001	15-JUL-26	45000	Delivered
2507	C-2002	10-AUG-26	75000	Shipped
2508	C-2003	22-AUG-26	120000	Pending
2509	C-2004	05-SEP-26	55000	Delivered
2510	C-2005	18-SEP-26	95000	Shipped
2501	C-2001	12-MAR-26	9000	Shipped
2502	C-2002	21-JUN-26	80000	Delivered
2503	C-2003	21-SEP-26	100000	Delivered
2504	C-2004	18-JAN-26	300000	Pending
2505	C-2005	27-MAY-26	250000	Shipped

```
10 rows selected.
```

```
SQL> SELECT * FROM ORDER_DETAILS;
```

ORDER_DETA	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
0104	2504	P2004	10	5000
0105	2505	P2005	12	3500
0106	2506	P2006	20	1800
0107	2507	P2007	5	6500
0108	2508	P2008	15	1200
0109	2509	P2009	6	7500
0110	2510	P2010	10	2200
0101	2501	P2001	15	28500
0102	2502	P2002	25	3499
0103	2503	P2003	8	42000

```
10 rows selected.
```

10. CRUD Operations

The Orders table supports basic CRUD operations:

- **Create:** A new customer order record was added.
- **Read:** Order records were retrieved and displayed.
- **Update:** The order status or total amount was modified.
- **Delete:** An order record was removed.

```
SQL> INSERT INTO ORDERS VALUES ('2511','C-2001',TO_DATE('2026-09-26','YYYY-MM-DD'),5000,'Pending');
```

```
1 row created.
```

```
SQL> SELECT * FROM ORDERS;
```

ORDER_ID	CUSTOMER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
2506	C-2001	15-JUL-26	45000	Delivered
2507	C-2002	10-AUG-26	75000	Shipped
2508	C-2003	22-AUG-26	120000	Pending
2509	C-2004	05-SEP-26	55000	Delivered
2510	C-2005	18-SEP-26	95000	Shipped
2501	C-2001	12-MAR-26	9000	Shipped
2502	C-2002	21-JUN-26	80000	Delivered
2503	C-2003	21-SEP-26	100000	Delivered
2504	C-2004	18-JAN-26	300000	Pending
2505	C-2005	27-MAY-26	250000	Shipped
2511	C-2001	26-SEP-26	5000	Pending

```
11 rows selected.
```

```
SQL> UPDATE ORDERS SET Order_Status='Delivered' WHERE Order_ID='2511';
```

```
1 row updated.
```

```
SQL> DELETE FROM ORDERS WHERE Order_ID='2511';
```

```
1 row deleted.
```

11. Generate Customer Order History Reports

Report 1: Customer Order History

The Customer Order History report displays the customer name, order ID, order date, total amount, and order status. This report helps to track the orders placed by each customer.

```
SQL> SELECT c.FIRST_NAME AS CUSTOMER_NAME, o.ORDER_ID, o.ORDER_DATE, o.TOTAL_AMOUNT, o.ORDER_STATUS FROM CUSTOMER c JOIN ORDERS o ON c.CUSTOMER_ID = o.CUSTOMER_ID;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
Lira	2506	15-JUL-26	45000	Delivered
Kenny	2507	10-AUG-26	75000	Shipped
Kairo	2508	22-AUG-26	120000	Pending
Suxi	2509	05-SEP-26	55000	Delivered
Blissy	2510	18-SEP-26	95000	Shipped
Lira	2501	12-MAR-26	9000	Shipped
Kenny	2502	21-JUN-26	80000	Delivered
Kairo	2503	21-SEP-26	100000	Delivered
Suxi	2504	18-JAN-26	300000	Pending
Blissy	2505	27-MAY-26	250000	Shipped

10 rows selected.

Report 2: Product-wise Order Report

The Product-wise Order Report identifies the number of times each product was ordered and the total quantity sold. This report helps to analyse product demand.

```
SQL> SELECT p.PRODUCT_NAME, COUNT(od.ORDER_ID) AS TIMES_ORDERED, SUM(od.QUANTITY) AS TOTAL_QUANTITY_SOLD FROM PRODUCT p JOIN ORDER_DETAILS od ON p.PRODUCT_ID = od.PRODUCT_ID GROUP BY p.PRODUCT_NAME;
```

PRODUCT_NAME	TIMES_ORDERED	TOTAL_QUANTITY_SOLD
Brake Pad Set	1	15
Castrol Engine Oil	1	25
Car Phone Holder	1	8

PRODUCT_NAME	TIMES_ORDERED	TOTAL_QUANTITY_SOLD
Bike Crash Guard	1	10
Steel Helmet	1	12
Exide Car Battery	1	20

Report 3: Customer Purchase Analysis

The Customer Purchase Analysis report analyses customer ordering activity. It identifies customers with maximum orders, calculates total spending by each customer, and determines the average order value.

```
SQL> SELECT c.FIRST_NAME AS CUSTOMER_NAME, COUNT(o.ORDER_ID) AS MAXIMUM_ORDERS, SUM(o.TOTAL_AMOUNT) AS TOTAL_SPENDING, AVG(o.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM CUSTOMER c JOIN ORDERS o ON c.CUSTOMER_ID = o.CUSTOMER_ID GROUP BY c.FIRST_NAME ORDER BY MAXIMUM_ORDERS DESC;
```

CUSTOMER_NAME	MAXIMUM_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
Lina	2	54000	27000
Kenny	2	155000	77500
Blissy	2	345000	172500
Suxi	2	355000	177500
Kairo	2	220000	110000

12. Conclusion

The Customer Order Management System was successfully designed and implemented for the Auto Hub automobile accessories and spare parts shop. The Orders and Order_Details tables were created, primary key and foreign key relationships were established, and necessary constraints were applied.

Customer order and product-wise order details were inserted and used for CRUD operations and customer purchase reporting. The generated reports help to track customer orders, analyse product sales, and understand customer purchasing activity.